

Contents

EmoFlow 项目算法详解：从 0 基础到完整数学推导	1
0. 阅读指南	1
第 1 章—EmoFlow 项目全景	1
第 2 章—数学基础铺垫（0 基础必读）	2
第 3 章—模块 1: StimulusEncoder（语义编码器）	4
第 4 章—模块 2: TemporalMemory（时序记忆）	7
第 5 章—模块 3: BayesianHead（贝叶斯融合头）	9
第 6 章—模块 4: 输出层与 Multilabel 重构	10
第 7 章—训练流程	12
第 8 章—Sigmoid 饱和与梯度消失（项目最大的 Debug 故事）	13
第 9 章—与 ECS 271 syllabus 概念的对照映射	16
附录 A 一公式速查表	17
附录 B 一关键超参数（来自实际项目）	17
附录 C 一推荐学习路径	17

EmoFlow 项目算法详解：从 0 基础到完整数学推导

ECS 271 Final Project 一适合零基础读者的逐步讲解
作者整理：覆盖项目中所有核心算法、结构、几何含义与数学推导

0. 阅读指南

本文档按“知识树”组织：

1. 第 1 章 一鸟瞰：EmoFlow 在做什么、整体长什么样
2. 第 2 章 一数学/统计基础铺垫（向量、概率、softmax、sigmoid、神经网络）
3. 第 3 章 一模块 1: StimulusEncoder（Transformer + LoRA + AppraisalHead）
4. 第 4 章 一模块 2: TemporalMemory（指数衰减 attention）
5. 第 5 章 一模块 3: BayesianHead（贝叶斯融合）
6. 第 6 章 一模块 4: 输出层（Multilabel + Sigmoid + BCE）
7. 第 7 章 一训练：联合损失、优化、阈值
8. 第 8 章 一核心 debug 故事：Sigmoid 饱和与梯度消失（这是本项目最大的“机器学习教训”）
9. 第 9 章 一与 ECS 271 syllabus 概念的对照映射

每一节遵循同一模板：- 是什么 / 为什么一直观解释 - 结构 / 公式一形式化定义 - 几何理解一让公式“看得见” - 数学推导——一步一步走完 - 统计 / 数学原理补充一为 0 基础读者讲清楚里面用到的预备知识

第 1 章—EmoFlow 项目全景

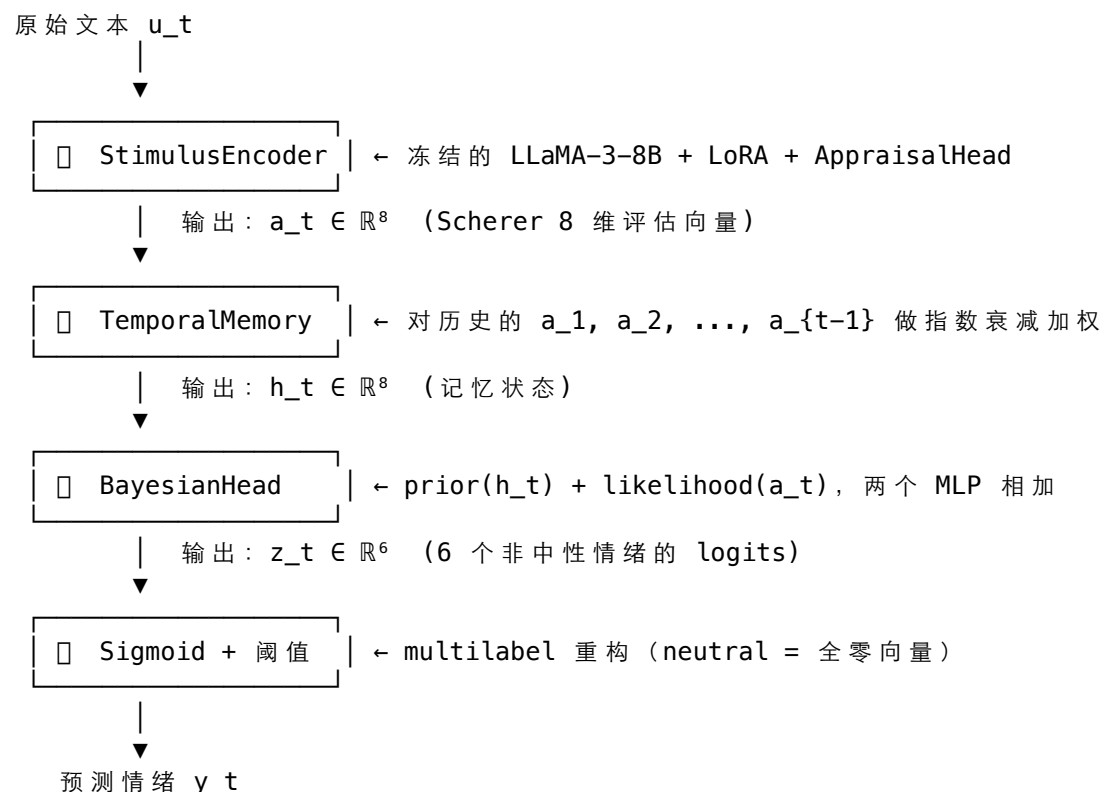
1.1 任务定义（用人话说）

给一段多人对话（比如 Friends 剧集字幕），每一句话标出说话人当前的情绪是哪一种：neutral / joy / sadness / anger / fear / disgust / surprise（7 类）。

这个任务叫 Conversational Emotion Recognition (ERC)。它和单句情绪分类不同的地方在于：

- 上下文依赖：“Sure.” 既可以是中性认同，也可以是嘲讽（disgust），也可以是惊讶（surprise）。只有看了前面的几句话才能判断。
- 情绪惯性：人发完三次怒，第四句即便平淡也大概率仍处于怒中。
- 严重的类别不平衡：MELD 数据集里 48% 的句子是 neutral，而 fear + disgust 加起来不到 5%。

1.2 EmoFlow 的”五件套”流水线



1.3 一句话总结

EmoFlow 的核心思想: 让 LLM 提取”心理学意义上的评估维度”, 然后用贝叶斯推理把”上下文记忆”和”当前证据”融合起来, 得到情绪概率。

接下来我们逐个零件拆开看。

第 2 章—数学基础铺垫 (0 基础必读)

2.1 向量、矩阵、内积

向量 就是一串数字, 比如 $a = [0.5, -0.3, 1.2]$ 。在三维空间里就是一个箭头, 从原点指向 $(0.5, -0.3, 1.2)$ 。

矩阵 就是一堆数字排成方阵或长方阵。它最关键的角色是: 矩阵作用在向量上, 把向量”线性变换”成另一个向量。

$$y = W \cdot x$$

如果 W 是 $d \times k$ 矩阵, x 是 k 维向量, 那么 y 是 d 维向量。几何上, W 把 x 进行了”旋转 + 拉伸 + 投影”。

内积 (点积) $a \cdot b = a_1b_1 + a_2b_2 + \dots + a_nb_n$ 。

几何意义: $a \cdot b = |a| \cdot |b| \cdot \cos(\theta)$, 其中 θ 是 a 和 b 的夹角。

关键直觉: 内积衡量两个向量”方向上有多像”。

2.2 神经元 = 加权 + 激活

一个最简单的神经元做的事:

$z = w \cdot x + b$ ← 线性加权求和（这就是“线性模型”）
 $y = \sigma(z)$ ← 激活函数（让模型能学非线性）

- w 是权重向量（“我应该给每个特征多大重视”）
- b 是偏置（“我天生倾向于哪边”）
- σ 是激活函数：sigmoid、ReLU、GELU 等等

几何理解： $w \cdot x + b = 0$ 在 $\mathbb{R}^n$ 中是一个超平面（2D 里是直线，3D 里是平面，更高维就是超平面）。这个神经元在干的事：判断 x 在超平面的“哪一侧”。

2.3 多层感知机 (MLP)

把神经元堆起来：

$h_1 = \sigma(W_1 x + b_1)$ ← 第一层
 $h_2 = \sigma(W_2 h_1 + b_2)$ ← 第二层
 $y = W_3 h_2 + b_3$ ← 输出层

几何理解：每一层都把空间“折”一下（激活函数提供非线性）。多层堆叠 → 可以拟合任何足够光滑的函数（通用近似定理 Universal Approximation Theorem）。

EmoFlow 里所有“head”（AppraisalHead、prior_head、likelihood_head）都是 MLP，结构都是：

Linear(d_in, d_hidden) → 激活函数 → Dropout → Linear(d_hidden, d_out)

2.4 概率与对数概率

概率 $P(e)$ 是 0 到 1 之间的数。对于一个事件集合 $\mathcal{E} = \{e_1, \dots, e_n\}$ ，要求 $\sum P(e_i) = 1$ 。

对数概率 $\log P(e)$ 。为什么用对数？两个原因：

1. 数值稳定：概率乘积 $P(a) \cdot P(b) \cdot P(c) \cdot \dots$ 很小，会下溢出。但对数化后变成 $\log P(a) + \log P(b) + \log P(c) + \dots$ ，加法不会下溢出。
2. 解析便利：乘法 → 加法，幂 → 乘法。求导也更方便。

2.5 Softmax 函数（一定要看懂）

把任意一组实数（叫 logits）变成合法的概率分布：

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

性质：- 输出非负（exp 是正的）- 输出加起来等于 1 - “放大差距”：原来 z 差 1，softmax 后概率比是 $e^1 \approx 2.7$

几何理解：softmax 把 $\mathbb{R}^n$ 空间映射到一个叫概率单纯形 (probability simplex) 的几何体里。在 3 类的情况下，这个单纯形就是 $(1,0,0)$, $(0,1,0)$, $(0,0,1)$ 三个顶点连成的三角形。

Softmax 的导数（求梯度时会用到）：

设 $p_i = \text{softmax}(z_i)$ ，则

$$\frac{\partial p_i}{\partial z_j} = \begin{cases} p_i(1 - p_i) & i = j \\ -p_i p_j & i \neq j \end{cases}$$

或写成矩阵形式： $J = \text{diag}(p) - pp^T$ 。

2.6 Sigmoid 函数

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

性质：- 把任意实数映射到 $(0, 1)$ - 在 $z = 0$ 处值为 0.5，导数最大 (= 0.25) - $z \rightarrow +\infty, \sigma \rightarrow 1$; $z \rightarrow -\infty, \sigma \rightarrow 0$ (“饱和”)

导数 (反复要用!):

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

关键观察：当 $\sigma(z)$ 趋近 0 或 1 时， $\sigma'(z) \approx 0$ 。这就是后面要讲的”sigmoid 饱和导致梯度消失”的根源 (第 8 章的核心)。

几何理解：sigmoid 是一条 S 形曲线。z 在 $[-3, 3]$ 这个”线性区”里梯度还行； $z > 5$ 或 $z < -5$ 就基本”贴着”上下平台，梯度几乎为 0。

2.7 损失函数与梯度下降 (训练机器学习模型的两块基石)

损失函数 $L(\theta)$ ：衡量当前模型参数 θ 下，预测和真实标签差多少。我们要最小化它。

两类最常见的：

- 均方误差 (MSE)：预测连续值时用。 $L = (y - \hat{y})^2$ 。几何上是抛物线，最小值在 $y = \hat{y}$ 。
- 交叉熵 (Cross-Entropy)：预测概率分布时用。 $L = -\sum y \log \hat{p}$ 。后面详讲。

梯度下降：

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} L$$

η 是学习率。直观：损失函数像个山谷，沿着”下坡方向” (梯度的反方向) 走一步就更接近谷底。

反向传播 (Backpropagation) = 高效计算 ∇L 的算法，本质就是链式法则。

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial h} \cdot \frac{\partial h}{\partial w}$$

如果中间任意一个偏导是 0 (比如 sigmoid 饱和)，整条链就乘出 0，参数无法更新。这就是梯度消失。

第 3 章一模块 1: StimulusEncoder (语义编码器)

目的：把一句自然语言文本 u_t 编码成一个 8 维的 Scherer 评估向量 $a_t \in \mathbb{R}^8$ 。

组件：1. 冻结的 LLaMA-3-8B (预训练 Transformer) 2. LoRA 适配器 (少量可训练参数) 3. 4-bit 量化 (QLoRA) 4. AppraisalHead (一个 MLP，把 4096 维投影到 8 维)

3.1 子模块 A: Transformer 与 Self-Attention

Transformer 是 LLaMA 的底层架构。它的核心机制是 Self-Attention (自注意力)。

3.1.1 Attention 在做什么 (直觉) 设你有一段话的 token 嵌入： $x_1, x_2, \dots, x_{\square}$ ，每个 $x_{\square} \in \mathbb{R}^{\square}$ 。

Self-attention 让每个 token “查询”其他所有 token，并按相似度加权求和。结果是：每个位置的新表示融合了”它跟谁有关”的信息。

3.1.2 数学公式 对每个 token，先用三个权重矩阵投影成三个向量：

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

- q: Query (“我想找什么”)
- k: Key (“我是什么”)
- v: Value (“我的内容是什么”)

然后注意力权重：

$$\alpha_{ij} = \frac{\exp\left(\frac{q_i \cdot k_j}{\sqrt{d_k}}\right)}{\sum_l \exp\left(\frac{q_i \cdot k_l}{\sqrt{d_k}}\right)}$$

这是个 softmax，分母对所有位置归一化。

最后输出：

$$o_i = \sum_j \alpha_{ij} \cdot v_j$$

3.1.3 几何理解 $q \cdot k$ 是内积 $\rightarrow$ 衡量”这两个 token 方向上多像”。

除以 $\sqrt{d_k}$ 是为了避免 d_k 大时内积值过大、导致 softmax 极端化。

softmax 把相似度变成”我对谁分多少注意力”的概率分布。

最后输出 o_i 是 values 的加权平均，权重就是注意力。

几何上： o_i 在所有 v_j 张成的凸包内（因为是凸组合）。它是”附近相关 tokens 的代表点”。

3.1.4 为什么这是革命性的？

- RNN 一步一步走，长距离信息会衰减（前面学过的 vanishing gradient）。
- Attention 是 $O(1)$ 的”任意位置相互访问”，长距离依赖更容易学。
- 可以完全并行计算（不像 RNN 必须顺序处理），训练大模型成为可能。

LLaMA-3-8B 就是把这种 attention 堆了 32 层，每层 32 个 head，预训练在 15T tokens 上。

3.2 子模块 B: LoRA (Low-Rank Adaptation)

这是 EmoFlow 适配 LLaMA 的核心技术，也是 ECS 271 syllabus 中”Adaptation”一章的标志性方法。

3.2.1 问题：为什么不能直接 fine-tune？ LLaMA-3-8B 有 80 亿参数。如果直接全量微调：- 训练时要存储所有参数的梯度 + 优化器状态（AdamW 需要 2 倍参数量的额外内存）- 一份完整的微调模型 = 一个 80 亿参数的副本（部署成本高）- 容易过拟合小数据集（MELD 只有 ~10000 句子）

3.2.2 LoRA 的核心假设 论文 (Hu et al. 2021) 的洞见：微调时权重的”改变量” ΔW 可能本质上是低秩 (low-rank) 的。

什么叫低秩？

矩阵的秩 (rank) = 它行（或列）张成的空间的维度。

一个 $d \times d$ 矩阵最多秩为 d （满秩）。但很多自然出现的矩阵秩远小于 d 。

比如 ΔW 是 $4096 \times 4096 = 1670$ 万参数。如果它的秩只有 8，那它实际上等价于：

$$\Delta W = BA, \quad B \in \mathbb{R}^{4096 \times 8}, \quad A \in \mathbb{R}^{8 \times 4096}$$

总参数： $4096 \times 8 + 8 \times 4096 = 65,536$ 。压缩比 255 倍。

直觉：高维空间里，任务相关的方向其实只有少数几个。其他方向上，预训练模型已经够好了不需要动。

3.2.3 LoRA 的具体实现 冻结原始权重 W_0 ，并行加入两个可训练矩阵 A, B ：

$$h = W_0 x + \Delta W x = W_0 x + BAx$$

初始化：- A 用高斯初始化 $\sim \mathcal{N}(0, \sigma^2)$ - B 初始化为 0

这样训练刚开始时 $\Delta W = BA = 0$ ，等价于原模型，保证训练稳定。

加一个缩放因子 α/r ：

$$h = W_0 x + \frac{\alpha}{r} BAx$$

EmoFlow 用 $r = 8, \alpha = 16$ ，applied 到 q_proj 和 v_proj 上。

3.2.4 几何理解

- W_0 把输入从一个 4096 维空间映到另一个 4096 维空间，是高维线性变换。
- $\Delta W = BA$ ：A 先把 $\mathbb{R}^{4096}$ 压到 $\mathbb{R}^8$ （降维投影，保留 8 个“任务相关方向”），B 再从 $\mathbb{R}^8$ 拉回 $\mathbb{R}^{4096}$ （重新展开）。
- 整体效果：在原变换基础上，只沿 8 个特定方向加微调。

3.2.5 参数量对比

方案	可训练参数	备注
全量微调 LLaMA-3-8B	80 亿	显存爆炸
LoRA ($r=8, q+v$)	~13 M	0.16%
EmoFlow 全部可训练	~20 M	LoRA + AppraisalHead + Memory + Bayes

好处：1. 显存友好（梯度只算 0.25% 的参数）2. 部署轻便（每个任务只需保存几十 MB 的 A, B ）3. 没有推理延迟：部署时可以预计算 $W = W_0 + BA$ ，跟原模型一样快

3.3 子模块 C：QLoRA（4-bit 量化 + LoRA）

LLaMA-3-8B 的 fp16 权重要 16 GB 显存。QLoRA 把冻结的 W_0 量化到 4-bit (NF4 格式)，显存降到 4 GB，但训练时 LoRA 的 A, B 仍用 bf16 精度。

为什么不影响精度？因为只有冻结的“前向计算用的权重”被量化了，反传梯度通过 LoRA 矩阵流动，仍是高精度。

3.4 子模块 D：AppraisalHead

LLaMA 输出每个 token 一个 4096 维向量。EmoFlow 取最后一个非 padding token 的向量 $h \in \mathbb{R}^{4096}$ 作为整句的代表（pooling），然后经过一个 MLP 投影到 8 维：

```
self.head = nn.Sequential(
    nn.Linear(4096, 4096),
    nn.GELU(),
    nn.Linear(4096, 8),      # ← 注意：没有 sigmoid! (这是关键 bug 修复)
)
```

输出 $\mathbf{a}_t \in \mathbb{R}^8$ 是 Scherer 心理学家定义的 8 个评估维度的值：

维度	含义
expectedness	这事预期到了吗？
unpleasantness	这事愉快还是不愉快？
goal_hindrance	这事妨碍我的目标吗？
external_causation	是外部原因造成的吗？
coping_potential	我能应付得来吗？
unfairness	不公平吗？
immorality	违反道德吗？
self_consistency	跟我对自己的认知一致吗？

每种情绪有一个”评估指纹”。比如：- 喜悦 (Joy): expectedness 高、unpleasantness 低、coping_potential 高、self_consistency 高 - 愤怒 (Anger): unpleasantness 高、goal_hindrance 高、unfairness 高 - 恐惧 (Fear): expectedness 低、unpleasantness 高、coping_potential 低

3.4.1 GELU 激活函数（顺便讲一下）

$$\text{GELU}(x) = x \cdot \Phi(x)$$

其中 Φ 是标准正态分布的累积分布函数 (CDF)。直观上：当 x 很大时近似 x ，当 x 很小（负）时近似 0，过渡平滑。它比 ReLU 更”软”，深度模型里效果通常更好。Transformer 时代以来基本是默认选项。

3.5 整个 Encoder 的几何理解（直觉总结）

LLaMA 拿到一句话后，把它通过 32 层 attention 不断地”重新组合 token 之间的信息”。最后一层得到的 4096 维向量已经隐含了句子的丰富语义。LoRA 微调让这个 4096 维空间里 8 个方向”重新调整”成”对情绪评估更敏感”的方向。最后 AppraisalHead 沿着这 8 个方向”读数”，输出 Scherer 的 8 维评估向量。

整个 Encoder 可看成一个：

$$f_{\theta} : \text{文本} \rightarrow \mathbb{R}^8$$

是一个高维”语义压缩器”。

第 4 章—模块 2: TemporalMemory (时序记忆)

目的：把过去所有句子的 appraisal 向量 $\mathbf{a}_1, \dots, \mathbf{a}_{\{t-1\}}$ 综合成一个记忆状态 $\mathbf{h}_t \in \mathbb{R}^8$ ，作为当前判断的”上下文先验”。

4.1 为什么不直接用 RNN/LSTM?

EmoFlow 实验里 BiLSTM 反而比”没有记忆”还差 14 个 wF1 点 (0.4241 vs 0.5631)。原因可能是：MELD 数据集小 (~10000 句)，LSTM 的”通用循环先验”容易过拟合到对话的偶然模式。

EmoFlow 采用了一种结构化、可解释、参数极少（只有 1 个参数!）的归纳偏置：指数衰减加权平均。

4.2 指数衰减的数学

$$h_t = \sum_{i \leq t} w_{t,i} \cdot a_i, \quad w_{t,i} = \frac{\exp(-\lambda(t-i))}{\sum_{j \leq t} \exp(-\lambda(t-j))}$$

其中 $\lambda \geq 0$ 是可学习的衰减率。

几何 / 直觉：- 当前句子 $i = t$ ：权重 $\propto e^0 = 1 \rightarrow$ 最大 - 上一句 $i = t-1$ ：权重 $\propto e^{-\lambda}$ - 再上一句：权重 $\propto e^{-2\lambda}$ - 距离越远权重越小，衰减速度由 λ 决定

极限情况：- $\lambda = 0$ ：所有句子权重相同 $\rightarrow$ 简单平均 - $\lambda \rightarrow \infty$ ：只看当前句 $\rightarrow$ 等价于无记忆 (stateless) - λ 适中：近的句子权重大，远的衰减

4.3 用 Softmax Attention 实现（代码效率技巧）

注意到分母正好是 softmax 的归一化项。实现时直接：

```
scores = -lambda * delta_t           # shape (T, T), 因果 mask 保证只看历史
weights = softmax(scores, dim=-1)    # 自动归一化
h = weights @ a                      # 加权求和
```

这复用了 PyTorch 高效的 softmax 实现，数值稳定（避免大数 exp 溢出）。

4.4 为什么 λ 要 “softplus” 参数化？

λ 必须 ≥ 0 （衰减必须非负）。但优化器是无约束的。所以引入：

$$\lambda = \text{softplus}(\theta) = \log(1 + e^\theta)$$

θ 是无约束的可学习参数。softplus 的输出永远 > 0 ，平滑可导。

4.5 几何理解：质心 (centroid) 视角

回忆 h_t 是 $a_1, \dots, a_t$ 的凸组合（权重非负且和为 1）：

$$h_t = \sum_i w_{t,i} \cdot a_i, \quad w_{t,i} \geq 0, \quad \sum_i w_{t,i} = 1$$

所以 h_t 永远落在 $\{a_i\}$ 张成的凸包内部。可以理解为： h_t 是历史 appraisal 点云的”加权质心”，重心偏向近期点。

4.6 实验结果

方案	wF1	解释
Stateless (无记忆)	0.5631	完全没上下文
BiLSTM	0.4241	通用 RNN，反而过拟合
EmoFlow $\lambda=0$ (均匀)	0.6052	均匀历史平均
EmoFlow λ 可学	0.6171	学到合适衰减

Takeaway：归纳偏置 (inductive bias) 的”形式”很重要。强结构、参数极少、可解释的设计能击败黑盒大模型，在小数据上尤其如此。

第 5 章—模块 3: BayesianHead (贝叶斯融合头)

目的: 把”当前句的 appraisal a_t ” 和”历史的记忆状态 h_t ” 融合成最终情绪的 logits z_t 。

结构: 两个独立的 MLP, 相加。

```
prior_logits      = prior_head(h_t)          #  $P(\text{emotion} \mid \text{历史})$ 
likelihood_logits = likelihood_head(a_t)      #  $P(\text{当前 appraisal} \mid \text{emotion})$ 
posterior_logits  = prior_logits + likelihood_logits
```

为什么是”相加”? 这是贝叶斯定理在 log 空间的体现。

5.1 贝叶斯定理回顾

设: - e = 情绪类别 (joy, sad, ...) - a_t = 当前 appraisal - $\mathcal{C}_t$ = 对话历史的上下文

我们想要的后验概率:

$$P(e \mid a_t, \mathcal{C}_t) = \frac{P(a_t \mid e) \cdot P(e \mid \mathcal{C}_t)}{P(a_t \mid \mathcal{C}_t)}$$

各项含义: - $P(e \mid a_t, \mathcal{C}_t)$: 后验—看了当前句和历史后, 每种情绪的概率 - $P(a_t \mid e)$: 似然—在某情绪下, 会产生这种 appraisal 模式的概率 - $P(e \mid \mathcal{C}_t)$: 先验—只看历史, 不看当前句, 每种情绪的概率 - $P(a_t \mid \mathcal{C}_t)$: 证据 (归一化常数)

5.2 取对数: 乘法变加法

两边取对数:

$$\log P(e \mid a_t, \mathcal{C}_t) = \log P(a_t \mid e) + \log P(e \mid \mathcal{C}_t) - \log P(a_t \mid \mathcal{C}_t)$$

最后一项与 e 无关 (对所有类别都一样), 是个常数。所以:

$$\log P(e \mid a_t, \mathcal{C}_t) = \underbrace{\log P(a_t \mid e)}_{\text{likelihood_logit}} + \underbrace{\log P(e \mid \mathcal{C}_t)}_{\text{prior_logit}} + \text{const}$$

重要观察: const 在后面 softmax/sigmoid 时会被消掉 (softmax 对常数偏移免疫)。所以模型输出 logits 时根本不需要算 partition function $\log Z$ 。

5.3 用神经网络实现

定义: - $\text{prior_head}(h_t) \approx \log P(e \mid \mathcal{C}_t)$, 输出 6 维 logits - $\text{likelihood_head}(a_t) \approx \log P(a_t \mid e)$, 输出 6 维 logits - $\text{posterior_logits} = \text{prior} + \text{likelihood}$

具体网络:

$\text{Linear}(8, 64) \rightarrow \text{GELU} \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(64, K=6)$

这是个标准的两层 MLP。两个 head 用相同结构但独立的参数。

5.4 为什么用两个 head 而不是一个大 head?

理论上 MLP[concat(h_t, a_t)] 表达能力更强。但两个 head 分开 + 相加有几个好处:

1. 可解释性 / 可控消融实验: 把 prior_head 设为零 → 测出”记忆 alone 的贡献”; 把 likelihood_head 设为零 → 测出”当前句 alone 的贡献”。
2. 归纳偏置: 强制模型遵循贝叶斯结构, 不让两路信息”自由勾兑”。这种显式结构假设在小数据上能防过拟合。
3. 训练稳定: 两个 head 的梯度互相独立, 不会相互”打架”。

实验中: Stateless (无 prior) wF1=0.5631, 加上 prior 后 EmoFlow 达到 0.6171, 贝叶斯分解贡献 +5.4 个点。

5.5 与 Dirichlet-Categorical 共轭的关系 (数学加餐)

提案里原本的方案是用 Dirichlet-Categorical 共轭对: - prior: Dirichlet(α) 分布 - likelihood: Categorical(p) 分布 - 后验更新: $\alpha_{\text{post}} = \alpha_{\text{prior}} + \text{counts}$

这种方法的对数形式恰好与”两个 logit 相加”等价 (在单纯形上、且边际分布均匀的假设下)。但 Dirichlet 在小 counts 时数值不稳, 所以最终选了 MLP-additive 实现。数学上等价, 工程上更稳。

5.5.1 什么是 Dirichlet 分布? (0 基础补课) Dirichlet($\alpha_1, \dots, \alpha_K$) 是”概率分布的分布”。它产生的样本是 k 维向量, 所有元素非负且和为 1 (即一个概率单纯形上的点)。

参数 α_K 大 → 对应类别概率倾向于大。所有 α 都大 → 分布集中 (高自信); 都小 → 分布弥散 (高不确定)。

Categorical 分布与 Dirichlet 是共轭的: 先验 Dirichlet + 似然 Categorical → 后验仍是 Dirichlet (参数只是把 counts 加上)。这就是为什么贝叶斯统计里 Dirichlet 用得得多。

第 6 章一模块 4: 输出层与 Multilabel 重构

6.1 原本的方案: 7 类 softmax + 交叉熵 (失败!)

最直接的做法:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^7 \exp(z_j)}, \quad k \in \{\text{neutral}, \text{joy}, \dots, \text{surprise}\}$$

损失: 分类交叉熵 (CCE)

$$\mathcal{L}_{CCE} = - \sum_{k=1}^7 y_k \log p_k$$

为什么失败?

MELD 里 48% 是 neutral。模型最简单的”投降策略”就是恒预测 neutral: - 对 neutral 样本, loss = $-\log p_{\text{neutral}} \approx 0$ (小) - 对其他样本, loss = $-\log p_{\text{neutral}} \approx 0.7$ (中等)

平均 loss 比”均匀预测”还低。这是个稳定的局部极小点, 类权重、label smoothing、oversampling 都打不破 (见第 8 章)。

6.2 Multilabel 重构: sigmoid + BCE

EmoFlow 的方案: - 词表去掉 neutral, 只剩 6 类: joy / sadness / anger / fear / disgust / surprise - 每类独立预测一个 sigmoid 概率 (不互斥) - Neutral = 全零向量 [0, 0, 0, 0, 0, 0] (没有任何情绪触发)

推理时:

```
p_k = sigmoid(z_k)
if max(p_k) < threshold τ:
    predict neutral
else:
    predict argmax_k(p_k)
```

$\tau = 0.2$ 是在 dev 集上调出来的。

6.3 为什么 Multilabel 打破了 collapse?

- 7 类 softmax: “全押 neutral” 是个输出无关的常数解 (只需让 z_{neutral} 远大于其他)。
- 6 类 multilabel: “预测 neutral” 需要让所有 6 个独立 logit 都被压到阈值以下——这要主动判别每种情绪都不存在, 不是“省事”的解。

直觉: multilabel 把“无情绪”这件事变成了一个需要积极论证的状态, 而不是一个默认状态。

6.4 BCE 损失 (二元交叉熵) 的推导

对每个独立的情绪 k :

设 $p_k = \sigma(z_k)$ 是预测概率, $y_k \in \{0, 1\}$ 是真实标签。

伯努利分布的对数似然:

$$\log P(y_k | p_k) = y_k \log p_k + (1 - y_k) \log(1 - p_k)$$

BCE 损失 = 负对数似然:

$$\mathcal{L}_{BCE} = -[y_k \log p_k + (1 - y_k) \log(1 - p_k)]$$

对 6 个 head 求平均:

$$\mathcal{L}_{BCE \text{ total}} = \frac{1}{K} \sum_{k=1}^6 -[y_k \log p_k + (1 - y_k) \log(1 - p_k)]$$

6.5 BCE 与 sigmoid 的优雅互动

复合 $\sigma + BCE$ 求关于 logit z 的梯度:

$$\frac{\partial \mathcal{L}_{BCE}}{\partial z} = \sigma(z) - y$$

惊人的简洁! 这恰好是“预测概率减去真实标签”。

推导 (自己跟着算):

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial z}$$

$$\frac{\partial \mathcal{L}}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p} = \frac{p-y}{p(1-p)}$$

$$\frac{\partial p}{\partial z} = p(1-p)$$

两者相乘：

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{p-y}{p(1-p)} \cdot p(1-p) = p-y \checkmark$$

$p(1-p)$ 被消掉了！这是为什么 **sigmoid + BCE** 是教科书标准组合：梯度形式简洁，不易消失（注意：这是输出层的 sigmoid，不是“中间层”的 sigmoid——后者就是第 8 章 bug 的根源）。

第 7 章—训练流程

7.1 联合损失

$$\mathcal{L} = \mathcal{L}_{BCE} + \alpha \cdot \mathcal{L}_{MSE}$$

$\alpha = 0.1$ 。

- $\mathcal{L}_{BCE}$ ：主任务（情绪分类）
- $\mathcal{L}_{MSE}$ ：辅助任务（让 AppraisalHead 输出匹配 Scherer Table 5.5 的目标向量）

MSE 损失公式：

$$\mathcal{L}_{MSE} = \frac{1}{|\mathcal{U}_a|} \sum_{u \in \mathcal{U}_a} \|a_u - t_u\|_2^2$$

其中 t_u 是该句对应情绪的 Scherer 目标向量。

几何理解：MSE 把 8 维 appraisal 空间中的点 a_u 拉向 t_u 。注意它不要求完全等于（只要“足够近”），所以模型有自由度去优化情绪分类。 $\alpha=0.1$ 让 MSE 起锚定作用但不主导。

7.1.1 MSE 损失的统计来源（0 基础补课） MSE 来自一个统计假设：预测误差服从高斯分布。

$$y = \hat{y} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

那么 y 的似然是：

$$P(y | \hat{y}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - \hat{y})^2}{2\sigma^2}\right)$$

取负对数：

$$-\log P(y) = \frac{(y - \hat{y})^2}{2\sigma^2} + \text{const}$$

最大化似然 $\leftrightarrow$ 最小化 $(y - \hat{y})^2 \leftrightarrow$ 这就是 MSE。

所以 MSE 损失隐含了“误差是高斯”的假设。这是个统计学结论。

7.2 类不平衡的两层防御

- 第一层：选择性跨数据集增强 (Selective Cross-Dataset Augmentation)。只引入 DailyDialog 中含有 fear / disgust / sadness 的对话，把 rare3 占比从 12.2% 提到 15.0%。
- 第二层：WeightedRandomSampler，在 dialogue 级别按“含稀有类的程度”加权采样。稀有 dialogue 被采到的概率高 30 倍。

7.3 优化：AdamW

AdamW 是 Adam 的一个变种，使用解耦权重衰减 (decoupled weight decay)。

Adam 更新规则（简化版）：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

(一阶动量)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

(二阶动量)

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t)$$

(偏差修正)

$$\theta \leftarrow \theta - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

直觉：Adam 给每个参数一个“自适应学习率”。梯度方差大的参数学慢，小的学快。AdamW 把权重衰减项 $-\eta \cdot \text{wd} \cdot \theta$ 直接加到参数更新里（而不是混在梯度里），效果通常更好。

EmoFlow 用 $\text{lr} = 5 \times 10^{-4}$, $\text{batch_size} = 2$ dialogues, $\text{epochs} = 3$ 。

7.4 阈值选择 τ

训练完成后，在 dev 上扫 $\tau \in \{0.2, 0.3, 0.4, 0.5, 0.6\}$ ，选最高 wF1。MELD 上 $\tau = 0.2$ 最佳。

为什么 τ 这么低？因为类别不平衡——稀有类的预测概率本来就偏低，需要降低门槛才能“看见”它们。

7.5 Dropout 与正则化

EmoFlow 各 head 都有 Dropout(0.1)。

Dropout 在做什么：训练时随机让 10% 的神经元输出变 0。为什么有效：相当于训练了无数个“子网络”的 ensemble，强迫每个神经元学到更鲁棒的特征，不依赖其他神经元的特定模式。

几何理解：每次前向传播都在一个“被截断的”网络上跑。模型必须在多个截断网络上都表现好，相当于对参数空间施加了 L2 类型的隐式正则。

第 8 章—Sigmoid 饱和与梯度消失（项目最大的 Debug 故事）

这一章是 EmoFlow 报告最有教学意义的部分。即便你不做这个项目，也要看完——它是理解深度学习训练失败模式的经典案例。

8.1 现象

最初的 AppraisalHead 设计是这样的：

```
self.head = nn.Sequential(
    nn.Linear(4096, 4096),
    nn.GELU(),
    nn.Linear(4096, 8),
    nn.Sigmoid(),          # ← 让输出在 [0, 1] 范围，匹配 Scherer 归一化目标
)
```

直观上这是“对的”：Scherer 表的 Z-scores 经过 min-max 归一化后在 [0, 1]，给输出加 sigmoid 看似很自然。

结果：模型崩溃了。- 训练 wF1 = 0.25 (瞎猜) - 所有 6 种“类不平衡 mitigation” (class weights、label smoothing、oversampling、pos_weight...) 都没用，得到 bit-identical 的 dev 指标

这种“完全相同”是诡异的信号：说明问题不在 loss，而在更上游——representation 完全没在学。

8.2 诊断步骤

8.2.1 神探技术 1：检查 encoder 输出的方差 把 3 句意思完全不同的话丢进 encoder，看 $\mathbf{a}_t \in \mathbb{R}^8$ 的输出：

```
Text 1 (joy):      [0.0001, 0.0002, 0.0001, ..., 0.0003]
Text 2 (anger):    [0.0001, 0.0002, 0.0001, ..., 0.0003]
Text 3 (sadness):  [0.0001, 0.0002, 0.0001, ..., 0.0003]
```

std across texts ≈ 0 ！不管输入是什么，输出几乎一样。这就是 representation collapse。

8.2.2 根因：Sigmoid 饱和 回忆 sigmoid 导数：

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

如果 z 大 (比如 $z > 5$)， $\sigma(z) \approx 1$ ，导数 ≈ 0 。如果 z 小 ($z < -5$)， $\sigma(z) \approx 0$ ，导数也 ≈ 0 。最大导数是 0.25 (在 $z = 0$ 时)。

链式法则的反向传播：

$$\frac{\partial L}{\partial W_{\text{linear}}} = \frac{\partial L}{\partial \sigma} \cdot \underbrace{\sigma'(z)}_{\approx 0} \cdot \frac{\partial z}{\partial W}$$

如果 σ 在饱和区，梯度被乘以 ~ 0 ，权重几乎不更新。信号无法从损失流回 encoder。

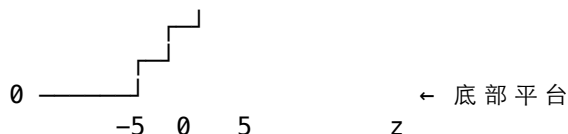
8.2.3 为什么会饱和？ LLaMA 输出的 4096 维向量经过 $\text{Linear}(4096, 4096) \rightarrow \text{GELU} \rightarrow \text{Linear}(4096, 8)$ ，最终 logits z 可能轻易达到 $|z| > 10$ 。

这时 sigmoid 把它们压到 0 或 1，梯度几乎消失。模型从一开始就被“卡”在饱和区，没法逃出来。

8.3 几何理解

把 sigmoid 想象成一根 S 形曲线：





预训练 LLaMA 输出经过两个 Linear 层后，分布通常很“宽”（大 $|z|$ ）。这意味着输出一开始就落在上下平台上——梯度近乎 0，根本学不动。

所有 6 种 imbalance 缓解策略都失败，因为它们都改的是“loss 端”的梯度（class weights 给某些类乘以 5, etc.），但乘以 5 还是 0 仍然是 0。问题在乘法链条上游！

8.4 修复：移除 sigmoid

```
self.head = nn.Sequential(
    nn.Linear(4096, 4096),
    nn.GELU(),
    nn.Linear(4096, 8),      # ← 输出是无界的  $\mathbb{R}^8$ 
    # 不再有 sigmoid
)
```

新担忧 1：“输出范围不再是 $[0, 1]$ ，跟目标 $[0, 1]$ 对不上怎么办？”

→ 没关系。MSE loss 本身就会把输出拉向目标。修复后实际输出大致落在 $[-0.5, 1.8]$ ，是个软约束而非硬约束。

新担忧 2：“输出会爆炸吗？”

→ 不会。BCE loss 提供反向压力，MSE 提供另一向的压力，两者会让输出收敛在合理范围。

8.5 修复后的诊断

Text 1: $[1.78, -0.35, -0.01, 0.14, 1.17, -0.20, 0.15, 1.21]$ ← joy 特征
 Text 2: $[0.16, 0.70, 0.70, 0.25, 0.18, 0.48, 0.48, -0.03]$ ← negative 特征
 Text 3: $[-0.43, 0.92, 1.09, 0.67, 0.61, 0.40, 0.36, -0.38]$ ← surprise 低 expectedness

mean std across texts: 0.413 (was 0.000)

encoder 在学了，而且学到了心理学上有意义的特征：- “I am so happy!” → coping_potential 高 + self_consistency 高 = Scherer 表上的 joy 指纹 - “Oh my god, what just happened?!” → expectedness 最低 = surprise 指纹 - “Get the fuck out!” → unfairness 高 + immorality 高 = anger 指纹

8.6 一般性教训

在低维 representation bottleneck 处避免使用有界激活函数

如果一定需要约束输出范围：- 错的做法：在网络里硬塞 sigmoid。会导致饱和、梯度消失。- 对的做法：让 loss 函数约束（MSE 把输出拉向目标范围）。如果非要 sigmoid，只在最终输出层使用（这时 sigmoid + BCE 会优雅地约掉 $p(1-p)$ ）。

8.7 调试 checklist（这一节教你 100 小时）

当分类指标崩塌时，按以下顺序排查：

1. 检查 encoder 输出的 std：用一批多样化的输入跑 encoder，看输出方差。 ≈ 0 = representation collapse。
2. 检查梯度幅度：在 backward 后看 `grad.abs().mean()`。极小 = 梯度消失。
3. 检查激活函数饱和：在 forward hook 里抓中间层值的分布。bunched at 0 or 1 = 饱和。
4. 不要假设 loss 是问题：如果上游 representation 都坏了，调 loss / weights / sampling 全是徒劳。

第 9 章一与 ECS 271 syllabus 概念的对照映射

Syllabus 主题	EmoFlow 里的对应
Linear Model	AppraisalHead 里的每个 Linear 层 ($W \cdot x + b$)
Logistic Regression	最终 sigmoid 输出 + BCE 损失 (每个情绪一个独立的逻辑回归)
Neural Networks	AppraisalHead, prior_head, likelihood_head 都是 MLP
Recurrent Neural Networks	LSTM baseline (实验里证明它反而不如指数衰减)
Learning Objectives / Regularization	联合 loss ($BCE + \alpha \cdot MSE$), Dropout(0.1), AdamW 的 weight decay
Self-supervised Learning	LLaMA-3-8B 的预训练 (next-token prediction 是 self-supervised)
Attention, Transformers	LLaMA 整个架构; TemporalMemory 也是 softmax-attention 实现
Pre-trained Transformers / Foundation Models	LLaMA-3-8B 作为冻结 encoder
Adaptation	LoRA + QLoRA! 这是项目最直接的 syllabus 映射
AI Safety / Robustness	sigmoid saturation 诊断 = “failure mode 诊断” 思维

9.1 EmoFlow 的”知识合成”特征

EmoFlow 是一个把多种 syllabus 概念有机组合的项目，不是单一算法：

心理学理论 (Scherer CPM)
 +
 预训练大模型 (LLaMA) ← Foundation Models
 +
 高效适配 (LoRA + QLoRA) ← Adaptation
 +
 结构化记忆 (exp decay attn) ← Attention
 +
 概率推理 (Bayesian) ← Logistic Regression + Cross-Entropy
 +
 多标签输出 (sigmoid + BCE) ← Logistic Regression
 +
 联合监督 (BCE + MSE) ← Learning Objectives
 +
 正则化 (dropout, weight decay) ← Regularization
 =
 EmoFlow

9.2 最重要的三个”工程教训”

1. 理论 grounding 能指导表示设计 —Scherer 的 8 个评估维度提供了一个可解释、抗 overfit 的 bottleneck。8 个数字浓缩情绪信息，比黑盒 256 维特征更有效。
2. 贝叶斯分解便宜地添加结构 —把 prior 和 likelihood 分开做，没多任何参数，但 +5 个 F1 点。这是”免费午餐”。
3. 先诊断 representation，再怪 loss —当分类崩塌时，第一步检查 encoder 输出有没有方差，而不是死调 class weights。Sigmoid 饱和是个隐蔽的杀手。

附录 A 一公式速查表

公式	用在哪
$\text{softmax}(z_i) = \exp(z_i) / \sum \exp(z_j)$	LLaMA attention, TemporalMemory
$\sigma(z) = 1 / (1 + e^{-z})$	输出层 multilabel
$\sigma'(z) = \sigma(z)(1-\sigma(z))$	反传梯度
$\text{BCE} = -[y \log p + (1-y) \log(1-p)]$	情绪分类主损失
$\text{MSE} = (1/n) \sum \ a_u - t_u\ ^2$	Appraisal 弱监督
$h = W_0 x + (\alpha/r) \text{BA}x$	LoRA
$w_{\{t,i\}} \propto \exp(-\lambda(t-i))$	TemporalMemory
$\log P(e a,C) = \log P(a e) + \log P(e C) - \log Z$	BayesianHead
$\theta \leftarrow \theta - \eta \cdot \nabla L$	梯度下降
$\theta \leftarrow \theta - \eta \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$	AdamW 更新

附录 B 一关键超参数（来自实际项目）

超参	值
LLaMA-3-8B 量化精度	NF4 (4-bit)
LoRA 秩 r	8
LoRA α	16
LoRA 应用位置	q_proj, v_proj
LoRA dropout	0.05
Appraisal 维度	8
BayesianHead 隐层	64
TemporalMemory λ 初值	0.1 (softplus 参数化)
Joint loss α	0.1
优化器	AdamW
学习率	5e-4
Batch size	2 dialogues
Epochs	3
输出阈值 τ	0.2
可训练参数总数	~20M (占模型 0.25%)

附录 C 一推荐学习路径

如果你想真正掌握这些概念，按此顺序学习（每个用 1-3 天）：

1. 基础：矩阵运算、概率论入门、贝叶斯定理 → 任何线性代数与概率教材
2. 机器学习：线性回归、逻辑回归、梯度下降 → Andrew Ng 的 ML 课
3. 深度学习：MLP、反向传播、激活函数 → CS231n 前 4 节
4. Transformer：attention、self-attention、multi-head → “The Illustrated Transformer” 博客 + 原论文
5. 预训练 & 微调：BERT / GPT、LoRA → LoRA 论文 + Hugging Face PEFT docs
6. 专题：贝叶斯方法、Dirichlet 共轭、softmax 的导数

全文完。这份文档应该足以让你完整理解 EmoFlow 项目的每一个零件——从 0 基础数学概念，到大模型适配的工程细节，再到训练失败的诊断逻辑。在准备 final presentation 或答辩时，建议至少把“为什么 sigmoid 饱和”、“LoRA 的低秩假设”、“贝叶斯分解为什么是相加”三件事讲到能自如脱稿。